

Aethernet: Implementation of a Full-Stack Audio Network

CS120 Project Report

ShanghaiTech University, School of Information and Science Technology
Student ID: 2023533090 2023533144

January 12, 2026

Abstract

In this project, we successfully engineered a complete network protocol stack, named "Aethernet," capable of transmitting digital data over audio signals using a standard microphone and speaker setup. The system was built using a hybrid architecture: C++ (utilizing the JUCE framework) was employed for the high-performance requirements of the physical and data link layers, while Python (leveraging Scapy) was used to implement complex logic for the network and application layers. The project progressed from transmitting simple chirps through the air to establishing a robust link over audio cables, implementing Carrier Sense Multiple Access (CSMA), and eventually constructing a fully functional IP stack. Key achievements include the implementation of a custom Network Address Translation (NAT) router capable of handling TCP/UDP traffic, a local DNS recursive resolver with traffic filtering, and a user-space TCP stack that successfully performs HTTP data retrieval. This report details the technical evolution, algorithmic choices, and the troubleshooting methodologies employed to overcome challenges such as audio latency, signal distortion, and operating system background noise.

1 Introduction

The fundamental objective of this course project was to demystify the abstractions of modern computer networks by building one from scratch. Modern networking stacks are typically hidden within the operating system kernel; by bringing these layers into user space and using audio as the transmission medium, we were forced to confront the physical realities of signal processing, error correction, and latency management.

Unlike the reference implementation which transitioned to Rust, our team opted for a hybrid approach. We utilized **C++ with the JUCE framework** for the real-time audio processing components (Physical and Link layers). C++ offers unparalleled control over memory and timing, which is critical when modulating bits into sound waves at 48kHz. For the upper layers (Network to Application), we utilized **Python**. Python's rapid prototyping capabilities and powerful libraries like **Scapy** allowed us to manipulate

IP headers, manage routing tables, and implement complex state machines (like TCP handshakes) with greater efficiency than lower-level languages would permit.

This project not only demonstrated the feasibility of transmitting Internet Protocol (IP) packets over sound waves but also highlighted the critical importance of error handling, timeouts, and protocol adherence in heterogeneous network environments.

2 Project Technical Details

2.1 Project 0: Environment and Architecture

The initial phase focused on establishing a stable development environment. We selected Windows as our primary operating system. To bridge the gap between the operating system's network stack and our audio modem, we utilized a **TAP adapter** (configured as "Aethernet"). This virtual network interface allowed us to intercept raw Ethernet frames generated by the OS and inject them into our custom stack.

On the audio side, we integrated the JUCE framework into a C++ console application. JUCE provided a cross-platform wrapper for ASIO and Windows Audio Session API (WASAPI), ensuring low-latency access to the input/output audio buffers. A critical architectural decision was the design of the **Producer-Consumer model** using thread-safe queues. One thread handled the high-priority audio callback (filling/reading sample buffers), while a separate worker thread handled the packet logic (framing, CRC calculation).

2.2 Project 1: The Physical Layer

The physical layer's responsibility was to convert digital bits into analog waveforms. We implemented a modulation scheme based on Frequency Shift Keying (FSK) concepts, representing bits as sequences of high and low amplitudes or specific wave patterns.

To ensure synchronization between the transmitter and receiver, we implemented a **Preamble detection** mechanism. A specific bit sequence (e.g., 0b10100101) served as a chirp signal. The receiver continuously sampled the audio input, running a correlation function or a state machine to detect this pattern. Only when the preamble was successfully identified would the system attempt to decode the subsequent payload.

We encountered significant noise issues in the air medium. To mitigate this, we implemented a **Cyclic Redundancy Check (CRC32)**. Every frame transmitted included a checksum. If the receiver's calculated checksum did not match the received checksum, the frame was discarded immediately. This ensured that corrupted data did not propagate up the stack.

2.3 Project 2: Data Link Layer and Reliability

With basic transmission working, Project 2 focused on reliability and medium access control. We introduced a **Stop-and-Wait ARQ (Automatic Repeat reQuest)** protocol.

- **Sender Logic:** Upon sending a data frame, the sender starts a timer (configured via `ACK_TIMEOUT`). If no Acknowledgment (ACK) is received before the timer expires, the frame is retransmitted. We tuned the `ACK_MAX_TRY` parameter to 10 and the timeout to 2.0 seconds to accommodate the slow nature of audio transmission.

- **Receiver Logic:** Upon successfully decoding a frame with a valid CRC, the receiver immediately generates and transmits a small ACK frame back to the sender.

To handle potential collisions (since audio is a shared medium), we implemented a basic **Carrier Sense (CSMA)** mechanism. Before transmitting, the system checks the current average amplitude of the microphone input. If the energy exceeds a threshold, the system assumes the channel is busy and defers transmission.

We also implemented a **MAC Payload structure**, encapsulating the Source ID, Destination ID, Sequence Number, and Type (Data vs. ACK) into the byte stream. This allowed us to support addressing, enabling Node 1 and Node 2 to distinguish traffic meant for them versus traffic meant for others.

2.4 Project 3: Network Layer and NAT

This phase marked the transition from C++ to Python for logic processing. The C++ program acted as a "modem," piping raw hex data to the Python script via standard I/O.

The Node 1 Client: The Python script on Node 1 sniffed packets from the TAP interface. It filtered for IP packets, stripped the Ethernet header, and forwarded the raw IP payload to the C++ modem. Conversely, incoming audio packets were wrapped in a fake Ethernet header (`FAKE_GATEWAY_MAC`) and injected back into the TAP adapter so the OS could recognize them.

The Node 2 Router (NAT Implementation): Node 2 acted as a gateway between the audio network (`172.18.x.x`) and the campus LAN (`10.20.x.x`). The core innovation here was the **Network Address Translation (NAT)** table.

- **Outbound (SNAT):** When Node 1 sent a packet to the Internet, Node 2 recorded the mapping (`Source IP, Source Port`) -> (`NAT ID`). It then replaced the source IP with its own WAN IP and forwarded the packet via the Wi-Fi interface. We initially used `sendp` (Layer 2) but later optimized this to use `send` (Layer 3) to allow the OS to handle ARP resolution for the upstream gateway automatically.
- **Inbound (DNAT):** When the Internet replied, Node 2 checked its NAT table. Using the destination port (or ICMP ID) as the key, it retrieved the original internal IP (Node 1) and port, modified the destination header, and forwarded the packet back over the audio link.

This implementation successfully supported ICMP Echo (Ping), allowing Node 1 to ping external servers like `8.8.8.8` through the audio link.

2.5 Project 4: Application Layer (DNS & HTTP)

The final phase introduced advanced networking features: Domain Name System (DNS) resolution and Hypertext Transfer Protocol (HTTP).

2.5.1 Task 1: DNS Recursive Resolver

We implemented a custom DNS server on Node 2. Instead of simple forwarding, we implemented a logic that intercepts UDP port 53 traffic.

1. **Optimization:** Initially, we attempted a full recursive lookup starting from Root Servers. However, due to latency, we optimized this to use system API calls on Node 2, acting as a forwarding proxy.
2. **IP Spoofing:** To satisfy the requirement of configuring the client DNS to 1.1.1.1, our Node 2 router intercepted packets destined for 1.1.1.1, resolved the domain locally, and then **spoofed the source IP** of the response to look like it came from 1.1.1.1. This tricked the Windows OS on Node 1 into accepting the DNS response.
3. **Traffic Filtering:** A "White-list" mechanism was implemented to filter out background noise. Windows constantly queries domains like `microsoft.com`. To prevent these from clogging the low-bandwidth audio link, we configured Node 2 to only resolve specific domains (e.g., `example.com`, `pku.edu.cn`) and silently drop others.

2.5.2 Task 2: HTTP and TCP Stack

For the HTTP task, we implemented a simplified user-space TCP stack in Python on Node 1.

- **Three-Way Handshake:** We manually constructed the TCP SYN packet with a specific Initial Sequence Number (ISN) format (e.g., `0x12345678`). We then waited for the SYN-ACK and replied with an ACK to establish the connection ESTABLISHED.
- **Data Transfer:** We constructed an HTTP GET request packet. Upon receiving the TCP segments containing the HTML payload, we reassembled the data.

3 Obstacles and Solutions

Throughout the development process, we encountered several critical issues that required deep debugging and engineering solutions.

3.1 The "Cold Start" Latency (ARP Issues)

Problem: We observed that when initiating a `ping` from Node 1, the first few packets were always lost, or the latency was extremely high ($> 2000\text{ms}$), while subsequent packets were fast.

Analysis: Investigation revealed two causes. First, the Windows OS was broadcasting ARP requests (*Who has 172.18.1.2?*) which our audio stack was ignoring. Windows would wait for a timeout before sending the actual IP packet. Second, the audio hardware entered a low-power state when idle, causing the preamble of the first packet to be truncated.

Solution:

1. **Static ARP Binding:** We used the command `netsh interface ip add neighbors` to hardcode the MAC address of Node 2 into Node 1's ARP table. This eliminated the ARP broadcast delay entirely.
2. **Audio Warm-up:** We modified the C++ sending function to prepend a short period of silence/noise before the actual preamble. This "woke up" the DAC/ADC hardware before the critical data was transmitted.

3.2 Gateway MAC Address Resolution

Problem: In Task 1/3, Node 2 could successfully resolve DNS and generate Ping packets, but the campus gateway refused to forward them, resulting in "Destination Host Unreachable."

Analysis: We initially hardcoded a generic Gateway MAC address. However, in the specific campus network subnet, the gateway MAC was different.

Solution: We switched our Python sending logic from Layer 2 (`sendp` with manual Ethernet headers) to Layer 3 (`send`). This allowed the Windows OS on Node 2 to handle the ARP resolution for the next-hop gateway automatically, ensuring the Ethernet destination MAC was always correct.

3.3 Audio Link Instability

Problem: During DNS testing, we frequently saw the error "*Critical Error: Max retries reached*" in the C++ logs.

Analysis: DNS response packets are significantly larger than simple ICMP packets. The probability of bit errors increases with packet length in the audio medium. The default ARQ parameters were insufficient.

Solution: We tuned the protocol parameters, increasing `ACK_MAX_TRY` to 10 and `ACK_TIMEOUT` to 2.0 seconds. We also implemented a rigorous checksum verification deletion (`del pkt[IP].chksum`) in the NAT logic to ensure Scapy recalculated valid checksums for modified packets.

4 Conclusion and Suggestions

This project was a rigorous exercise in full-stack network engineering. By building the stack from the bottom up, we gained an intuitive understanding of concepts that are often treated as abstract theory. We learned that the "Physical Layer" is not just a concept but a messy reality of sound waves, interference, and hardware latency. The transition from a simple "Hello World" audio transmission to a complex TCP/IP stack capable of browsing the web was challenging.

The use of C++ for the heavy lifting of signal processing combined with Python for high-level logic proved to be an effective strategy, balancing performance with development speed.

Suggestions for Future Iterations:

- **Standardize Audio Hardware:** The variability in microphone sensitivity caused discrepancies. Providing a standard USB audio interface could standardize the physical layer baseline.

- **Clearer Documentation on ARP:** The "Cold Start" issue related to ARP baffled us for a long time. Future course materials could explicitly mention the interaction between the OS ARP cache and the custom user-space stack.

References

- [1] JUCE Framework Documentation. <https://juce.com/>
- [2] Scapy Documentation. <https://scapy.net/>
- [3] RFC 793 - Transmission Control Protocol.
- [4] RFC 1035 - Domain Names - Implementation and Specification.
- [5] Course Project Documentation, Fall 2024.